VORLAGE FÜR DEN DATENSCHUTZBEAUFTRAGTEN

Externe Dienstleister und Datenwege

Ausschließlich das, was heute im Projekt ist — Abgeschaltetes und Ausgebautes ist entfernt. Erhoben aus dem Quellcode beider Repositories und am laufenden Server nachgemessen, nicht aus Verträgen.

STAND 17.09.2026 QUELLEN Repos `lymbe-ai` , `lymbe-ablaeufe` BETRIEBSSTAND gemessen auf `lymbe-prod` ERHOBEN VON Christian Gudermann

Wer davon Auftragsverarbeiter im Sinne von Art. 28 DSGVO ist, gehört zum Datenschutzbeauftragten. Dieses Dokument sagt nur, **was tatsächlich wohin fließt**.

---

0 Was zu klären ist

Dreizehn Punkte, die die Antwort auf die Frage „Welche Anbieter haben wir?“ verändern können oder eine Zusage betreffen. **Zehn davon sind am 17.09.2026 behoben und ausgerollt** — sie stehen weiter hier, grün markiert, mit dem, was jetzt gilt. Wer wissen will, was das Problem war, findet es darüber.

0.1 Profilbilder gehen bei jedem Seitenaufruf an Google BEHOBEN · 17.09.2026

Meldet sich jemand über Google oder GitHub an, speichert der Anmelde-Adapter die **fremde** Bildadresse in `User.image` . Die Avatar-Komponente rendert sie unverändert als `<img src>` — in Kopfleiste, Mitgliederliste und Posteingang.

Damit erhalten Google bzw. GitHub IP, User-Agent und Referrer **jedes Betrachters**, bei jedem Seitenaufruf — nicht nur beim Anmelden.

In der Produktionsdatenbank nachgezählt: von drei Benutzern mit hinterlegtem Bild zeigen **zwei** auf `lh3.googleusercontent.com` . Kein

theoretischer Fall. Abstellbar, indem das Bild bei der Anmeldung einmal übernommen und selbst gespeichert wird.

`lib/auth.ts (PrismaAdapter, keine profile()-Umschreibung) · components/platform/ui/Avatar.tsx:64`

> **Behoben am 17.09.2026.** Zwei Riegel: Google und GitHub liefern ihr Profil jetzt ohne Bildadresse — sie kommt gar nicht mehr in die Datenbank —, und beim Anmelden holt der Server das Bild einmal in den eigenen Speicher (`lib/profile/anmeldefoto.ts`). Zusätzlich zeigen alle vier Stellen, die ein Personenbild zeichnen, nur noch relative Adressen an. Die beiden gemessenen Zeilen in der Produktionsdatenbank sind geleert; das Foto kommt bei der nächsten Anmeldung zurück, dann aus dem eigenen Speicher.

`0.2` Zählpixel in eingehenden Mails funktionieren `BEHOBEN · 17.09.2026`

Das Admin-Postfach rendert den rohen Mailtext in einem `srcDoc` -iframe. Skripte sind gesperrt — kein `allow-scripts` , das ist richtig gelöst. **Bilder und CSS-Hintergründe sind es nicht:** es gibt keine Bildsperre und kein „Bilder anzeigen“.

Ein Zählpixel im Mailtext erhält damit IP und User-Agent des Bearbeiters und den Lesezeitpunkt. `referrerPolicy="no-referrer"` begrenzt nur den Referrer, nicht die Anfrage selbst. Im Chat ist genau dieser Weg bewusst zugenagelt (`markdown-safety`), im Postfach nicht.

`components/admin/email/MessageView.tsx:58–80`

> **Behoben am 17.09.2026.** Drei Schichten, weil keine allein trägt: ein Bereiniger baut aus dem Mail-HTML ein eigenes Dokument und schließt die Wege, an die eine Sicherheitsrichtlinie prinzipbedingt nicht herankommt (`preconnect` , `dns-prefetch` , `meta refresh` , `base href`) — dazu den unsichtbaren Klickfänger, einen Link mit `position:fixed` über der ganzen Mail, bei dem jeder Klick den Absender anruft. Darunter eine eigene Richtlinie als erstes Element im Kopf, darunter der Rahmen ohne `allow-scripts` . Geschlossen wird **nichts** geladen, auch nichts von uns selbst. Wer die Bilder sehen will, drückt auf „Bilder anzeigen" — dann holt sie **unser Server**, und der Absender sieht die Adresse unserer Maschine statt der des Lesers. Die Rohfassung bleibt unverändert in der Datenbank; bereinigt wird beim Lesen, nicht beim Empfangen. Fünf Umgehungen (Klammer im CSS-Zitat, `image-set()` , CSS-Escapes im Funktionsnamen, der Klickfänger und der `<title>` im Fließtext) kamen erst beim adversarischen Gegenlesen ans Licht und stehen jetzt als Testfälle.

`0.3` Stripe.js lädt, ohne dass jemand klickt `BEHOBEN · 17.09.2026`

`loadStripe()` steht auf **Modulebene** einer Client-Komponente. `js.stripe.com` lädt deshalb, sobald die Abrechnungsseite gerendert wird — Stripe bekommt IP, User-Agent und Gerätesignale und setzt eigene Cookies, auch wenn niemand etwas kauft.

Abstellbar, indem `loadStripe` erst im Klick-Handler aufgerufen wird.

`components/platform/billing/AvailablePlansCard.tsx:7, :18`

> **Behoben am 17.09.2026.** Nicht in den Klick-Handler verschoben, sondern ganz entfernt: das geladene Stripe-Objekt wurde nirgends benutzt — der Checkout entsteht serverseitig, der Browser folgt nur der zurückgegebenen Adresse. Entscheidend war ein zweiter Auslöser, den man leicht übersieht: schon der **Import** von `@stripe/stripe-js` hängt `js.stripe.com/v3` in die Seite. Import, Aufruf und das Paket selbst sind raus. Nebenbefund: der öffentliche Schlüssel wird beim Bauen des Images gar nicht durchgereicht, `loadStripe(undefined)` scheiterte also — und riss die bereits erzeugte Checkout-Sitzung mit sich. „Plan auswählen" war damit vermutlich schon vorher kaputt; die Änderung repariert es nebenbei.

0.4 Die Websuche geht an OpenAI direkt — und trägt auch die Werkstatt HOCH

Die Suchfunktion „Frag Lymbe“ ist im Code fest auf den OpenAI-Zugang verdrahtet. Über Azure ist das technisch nicht möglich — das eingebaute Websuch-Werkzeug gibt es nur in OpenAIs Responses-API.

Am 17.09.2026 auf dem Server nachgemessen: auch die Werkstatt läuft dort (`AI_PROVIDER_WERKSTATT=openai` , nicht `azure`). OpenAI ist damit für **zwei** Funktionen Empfänger, Microsoft für keine (A4, G1).

`lib/ai/web-suche.ts · docker/.env auf lymbe-prod`

0.5 Das Widget speichert auf den Websites unserer Kunden § 25 TDDDG

`widget.js` legt `lymbe_ignore` im localStorage ab und nutzt sessionStorage für die proaktive Ansprache. Beides ist Speichern auf dem Endgerät. Der Code argumentiert selbst, dass es um technisch Erforderliches geht — diese Einordnung sollte bestätigt werden, weil unsere Kunden sie in ihren eigenen Cookie-Hinweisen übernehmen müssen. Ein Textbaustein für sie fehlt.

`public/widget.js:507, :728, :783 ff.`

0.6 Die Ankündigungsfrist aus § 4 Abs. 4 AVV läuft FRIST

Die Liste der Unterauftragsverarbeiter auf `lymbe.ai/avv` nannte bis zum 17.09.2026 **einen** Eintrag. Seit den Deploys an diesem Tag nennt sie fünf (Fassung 1.2) — sechs waren es zwischenzeitlich, Cloudflare ist mit 0.13 am selben Tag wieder herausgefallen.

Rechtlich sind das keine neuen Dienstleister, sondern nachgetragene. Der Vertrag sagt Bestandskunden gleichwohl **14 Tage Vorlauf** zu, mit Widerspruchsrecht aus sachlichem Grund. Ob die Nachtragung diese Frist auslöst, gehört entschieden.

0.7 Drei Zusagen sind aus dem Code nicht belegbar BEHOBEN · 17.09.2026

Der AVV sagt einleitend, er werde „durch aktive Bestätigung (Checkbox im Onboarding-Prozess)“ geschlossen. **Diese Checkbox gibt es nicht.**

Damit fehlt der Nachweis, wann welcher Kunde welcher Fassung zugestimmt hat.

Die Hilfeseite führt zusätzlich auf: der AVV stehe „zum Download bereit“, es gebe ein Verzeichnis nach Art. 30 und eine „durchgeführte“ Datenschutz-Folgenabschätzung. Keines der drei ist aus dem Repository belegbar — entweder ablegen und verlinken, oder streichen.

> **Behoben am 17.09.2026.** Die Checkbox gibt es jetzt — beim Anlegen des Workspace, nicht bei der Anmeldung, weil der AVV zwischen Firmen geschlossen wird und nicht mit einem Benutzerkonto. Sie ist nicht vorausgewählt, der Knopf bleibt ohne sie gesperrt, und die Prüfung steht auch in der Server-Action: ein Aufruf am Formular vorbei hilft nicht. Jede Zustimmung wird als eigene Zeile abgelegt (Fassung, Zeitpunkt, Person, Adresse), in derselben Transaktion wie der Mandant. Die Fassung steht an genau einer Stelle im Code, damit der Vertrag und der Nachweis nie auseinanderlaufen. Die beiden unbelegbaren Zeilen — Art.-30-Verzeichnis und „durchgeführte“ DSFA — sind aus der Hilfeseite gestrichen; das sind Dokumente, keine Software, und sie dürfen zurück, sobald sie vorliegen. **Offen bleibt:** die vier Bestandsmandanten haben keinen solchen Nachweis, und rückwirkend einzutragen wäre das Gegenteil eines Nachweises — das gehört zusammen mit 0.6 erledigt.

`0.8` Gelöschte Widget-Dokumente bleiben als Datei liegen

`BEHOBEN · 17.09.2026`

Wird ein Widget-Dokument im Portal gelöscht, verschwindet die Datenbankzeile — die hochgeladene **Datei bleibt in MinIO**. Eine Löschung, die der Kunde auslöst, wird damit nicht vollständig vollzogen. Es fehlt ein Gegenstück zu `uploadFile()`.

`server/actions/widget-content-actions.ts · lib/storage/minio.ts`

> **Behoben am 17.09.2026.** Beim Löschen wird der Objektschlüssel aus der gespeicherten Adresse gelesen — beide gewachsenen Schreibweisen — und die Datei zuerst entfernt, dann die Zeile. Scheitert der Speicher, bleibt die Zeile stehen und der nächste Versuch findet sie wieder; andersherum wäre die Datei endgültig verwaist. Fremde Adressen (verlinkte Dokumente) werden ausgelassen. In den fünf Mandanten-Datenbanken der Produktion gibt es derzeit kein einziges solches Dokument — die Lücke war offen, aber leer.

`0.9` Zwei Fremdabrufe aus dem Browser, beide vermeidbar

`BEHOBEN · 17.09.2026`

Der Katalog „Apps verbinden“ lädt Baustein-Logos direkt von `cdn.activepieces.com` (D8), die Admin-Oberfläche des Shopify-Plugins lädt Polaris-CSS von `unpkg.com` (D9). Beides betrifft keine Websitebesucher, beides ließe sich durch Mitliefern abstellen.

> **Behoben am 17.09.2026.** Die Baustein-Logos laufen über `/api/bausteine/logo`: nur für Angemeldete, nur die Positivliste, nur Bilder, einen Tag zwischengespeichert — ein Abruf beim CDN je Logo und Tag statt einer je Seitenaufruf und Person. Der Polaris-Verweis im Shopify-Plugin ist ersatzlos entfernt; gebraucht wurde die Datei nie, die Seite trägt ausschließlich eigene Klassen.

0.10 Der Abläufe-Kern holt stündlich fremden Code nach BEHOBEN · 17.09.2026

`AP_PIECES_SYNC_MODE=OFFICIAL_AUTO`, am 17.09.2026 im Container nachgemessen: stündlich Katalog von `cloud.activepieces.com`, Pakete von `registry.npmjs.org`. Keine Kundendaten hinaus, wohl aber die Server-IP — und ausführbarer Code Dritter herein. Abschaltbar mit `NONE`.

> **Behoben am 17.09.2026.** `AP_PIECES_SYNC_MODE` steht auf `NONE`, im Protokoll des Kerns steht seither „Piece sync service is disabled“. Der Katalog in der Datenbank bleibt unverändert stehen, alles Verbundene läuft weiter — es kommt nur nichts Neues mehr von allein herein. Wer nachziehen will, stellt bewusst um, erneuert den Container und stellt zurück.

0.11 Wächter von außen fehlt BEHOBEN · 17.09.2026

Die Betriebsprüfung deckt fünf Dienste ab und meldet zuverlässig — aber sie läuft **in der App**, und die Meldung geht **über den eigenen Mailserver**. Fällt die Maschine ganz aus, fällt beides mit aus und niemand erfährt etwas.

> **Behoben am 17.09.2026.** `/api/puls` antwortet ohne Anmeldung mit ja/nein — und zwar erst nach einer Datenbankabfrage, denn Next.js sagt auch dann noch 200, wenn Postgres weg ist. Ein Lauf auf GitHubs Maschinen fragt alle 30 Minuten nach: Puls, Marketing-Seite, Restlaufzeit des Zertifikats. Gemeldet wird über einen Weg, der mit unserem Mailserver nichts zu tun hat. Zwei Grenzen stehen in der Datei: GitHub hält den Takt nicht auf die Minute, und nach 60 Tagen ohne Aktivität im Repository schaltet es den Zeitplan ab.

0.12 Abrufe an Adressen, die der Kunde bestimmt — mit Rückkanal BEHOBEN · 17.09.2026

Der Import einer Wissensquelle (C11) rief jede eingetragene Adresse **ungeprüft** ab — und legte die Antwort als durchsuchbares Wissen im Portal ab. Damit war das keine blinde Lücke, sondern eine mit Rückkanal: `http://inngest:8288/` eintragen, Import starten, Ergebnis im Chat lesen. Inngest läuft ohne Anmeldung, Qdrant und MinIO antworten auf HTTP; alle drei verlassen sich darauf, dass ihr Port nicht nach außen zeigt. Dieselbe Lücke im Website-Crawl, dort mit einer Prüfung, die `minio` und `qdrant` durchließ, weil punktlose Namen an ihrem `split('.').length === 4` scheiterten.

> **Behoben am 17.09.2026.** Ein gemeinsamer Riegel (`lib/abruf/`): die Adresse wird geprüft, **einmal** aufgelöst, und verbunden wird zu genau der geprüften Adresse — sonst wählt bei der zweiten Auflösung der Angreifer (DNS-Rebinding). Weiterleitungen geht der Server selbst, jede Station durch dieselbe Prüfung, und die entschlüsselten Zugangsdaten fallen beim Hostwechsel aus dem Kopf. Dazu Grenzen für Zeit, Größe (während des Lesens, nicht danach) und Typ. Umgestellt sind API-Import, Dokument-Neuaufbau und Crawler; die Shop-

Adresse wird beim Verbinden geprüft, und die Konnektoren zitieren fremde Antwortkörper nicht mehr in ihren Fehlermeldungen — das war der Weg, sich Antworten aus dem Docker-Netz vorlesen zu lassen.

Und ein Fund über den eigenen Code: die erste Fassung des Riegels war eine Attrappe. Seit Node 20 fragt der Verbindungsaufbau die Adressauflösung in einer anderen Form ab; die alte Form lieferte für jede Adresse mit einem *Namen* „nicht erreichbar" — aufgefallen wäre es erst beim Kunden, weil alle Tests gegen 127.0.0.1 liefen und ein IP-Literal die Auflösung gar nicht erst aufruft. Dieselbe Zeile stand in der Mail-Bildroute von 0.2: „Bilder anzeigen" hätte dort nie ein Bild geladen. Beides gemessen, behoben und mit einem Test festgehalten.

`0.13` Cloudflare abgelöst — und die Registrierung stand offen BEHOBEN · 17.09.2026

Cloudflare Turnstile war das **letzte fremde Skript auf den Websites unserer Kunden**: wurde dort ein Besucher auffällig, lud dessen Seite ein Skript von `challenges.cloudflare.com` und die Prüfung übertrug seine IP-Adresse in die USA. Für einen Schutz, der selten anspringt, war das ein Dienstleister im Verzeichnis, eine Zeile im AVV und die Frage, die der Datenschutzbeauftragte unseres Kunden stellt.

Abgelöst am 17.09.2026. Unser Server nennt eine Prüfsumme, der Browser sucht die Zahl, die sie ergibt (ALTCHA-Format, MIT-Lizenz, alles im Haus). Live gemessen: 696 ms in Chrome. **Was das Rätsel nicht ist:** eine Verhaltensprüfung. Turnstile hat gemessen, WIE sich jemand verhält; ein Rechenrätsel misst nur, DASS gerechnet wurde — wer nativ angreift, ist ein Vielfaches schneller als ein ehrlicher Browser. Es ist deshalb bewusst die letzte Stufe hinter den Ratenbremsen und nicht die erste.

Der Fund darunter: eine bestandene Prüfung war eine Freifahrt. Sie stand auf einem `return { ok: true }` und ging damit an Cooldown, Duplikat-, Burst-, Fünfminuten- und Tageslimit vorbei — die Anfrage wurde nicht einmal gezählt. Der Kreislauf „zweimal verstoßen, einmal lösen" lieferte beliebig viele ungezählte Modellaufrufe. Das bremste nur, solange eine Lösung bei Cloudflare Geld kostete. Jetzt kauft eine bestandene Aufgabe den Verstoßzähler frei, nicht das Kontingent.

Und die Registrierung, die nichts mit Cloudflare zu tun hat: sie hatte keine Ratenbremse (die Funktion dafür gab es, aufgerufen hat sie niemand), jede Anfrage kostete uns rund 100 ms Rechenzeit für das Passwort-Hashen, und die Antwort verriet, ob eine Adresse bei uns registriert ist — eine Auskunftsstelle darüber, wer Kunde ist. Jetzt fünf Registrierungen je Stunde und Anschluss, das Rätsel davor (das Formular rechnet es im Hintergrund, während getippt wird), die Passwort-Regel, und dieselbe Antwort in beiden Fällen; wem die Adresse gehört, erfährt es per Mail. Eine zweite Registrierungs-Route, die niemand aufrief, aber Konten ohne Bestätigungsmail anlegte, ist gelöscht. Am Login kam eine Obergrenze je Anschluss über alle Konten hinzu — die Fünfersperre je Konto stoppte kein Passwort, das gegen zehntausend Adressen läuft.

A Externe Empfänger personenbezogener Daten

Diese Anbieter sehen Inhalte, Stammdaten oder mindestens IP-Adressen. A1 bis A5 und A7 stehen seit dem 17.09.2026 auch in der Datenschutzerklärung und im AVV; A9 und A10 sind beim Nachmessen dazugekommen und dort noch nicht genannt. **A6 ist am selben Tag entfallen** — der Eintrag bleibt als Abgang stehen.

A1

26.lab.io (Christian Gudermann) DEUTSCHLAND

Technischer Betrieb der Plattform und der KI-Infrastruktur.

A2

Hetzner Online GmbH NÜRNBERG

Der gesamte Stack läuft auf **einem** Server: Anwendung, alle Datenbanken, Vektorspeicher, Dateispeicher, Abläufe-Kern und seit dem 16.09.2026 auch der Mailserver. Dazu die Hetzner-**Cloud**-API für Zertifikate und DNS-Einträge.

Gegen die Cloud-API gemessen: Typ `CPX32` , Standort `nbg1-dc3` („Nuremberg DC Park 1“). Die Maschine hat **zwei öffentliche Adressen** — eine fürs Portal, eine ausschließlich für Mail, damit der Ruf der Mail-Adresse unabhängig bleibt.

DATEN Alle. Hetzner ist der Betreiber der Maschine, auf der jede Verarbeitung stattfindet.

SITZ Industriestr. 25, 91710 Gunzenhausen

A3

Schwarz Digits Cloud GmbH & Co. KG — STACKIT DEUTSCHLAND

Der **zentrale Datenweg** und der mit Abstand größte: Chatverläufe, hochgeladene Dokumente, das hinterlegte Firmenwissen, E-Mail-Texte und Bilder. STACKIT liefert die Antworten *und* die Text-Embeddings für die Wissenssuche. Umgestellt am 03.09.2026.

Modelle: `Qwen3.8-27B` im Regelbetrieb, `Qwen3-VL-235B` für anspruchsvolle Fragen und Bilder, `gpt-oss-120b` für die Einordnung jeder Nachricht, `e5-mistral-7b-instruct` für die Embeddings.

Der Name `gpt-oss` führt in die Irre: das ist ein quelloffenes Modell, das STACKIT in Deutschland betreibt. OpenAI sieht davon nichts.

`SITZ` Bad Friedrichshall, HRA 741347 (AG Stuttgart)

A4

OpenAI Ireland Ltd. `WEBSUCHE + WERKSTATT`

Websuche „Frag Lymbe“. Hinaus geht ausschließlich der Suchbegriff, den das Modell formuliert hat — nicht das Gespräch, nicht die Dokumente, nicht wer gefragt hat. Dazu eine grobe Ortsangabe: fest verdrahtet Land `DE` . Verarbeitung ggf. USA.

Werkstatt für Abläufe (`gpt-5.6-sol`). Dorthin gehen Beschreibungstexte des Nutzers und die Ablaufdefinition — keine Kundengespräche. Zugangsdaten werden vorher maskiert. Verarbeitung im EU-Rechenzentrum, Schweden.

`ABSCHALTBAR` `OPENAI_API_KEY` leeren — die Suche entfällt, die Werkstatt fällt auf A3 zurück

A5

Stripe Payments Europe, Ltd. `IRLAND / USA`

Zahlungsabwicklung und Abonnementverwaltung. Kartendaten berühren unsere Systeme nicht.

Auch im Browser — siehe 0.3: `js.stripe.com` lädt beim Rendern der Abrechnungsseite. Checkout und Abrechnungsportal sind zusätzlich vollständige Weiterleitungen auf `checkout.stripe.com` bzw. `billing.stripe.com` .

`DATEN` Name, E-Mail, Rechnungsanschrift, ggf. USt-ID, Abo- und Zahlungsverlauf; im Browser zusätzlich IP und Gerätesignale

A6

Cloudflare Germany GmbH — nicht mehr beteiligt `GESTRICHEN · 17.09.2026`

Hier stand die progressive Bot-Prüfung „Turnstile" im Chat-Widget: der Browser des Websitebesuchers lud ein Skript von Cloudflare, die serverseitige Prüfung übertrug seine IP-Adresse. Es war das **einzige fremde Skript, das wir je auf Kundenwebsites geladen haben**.

> **Abgelöst am 17.09.2026 (0.13).** Die Prüfung stellt jetzt unser eigener Server als Rechenaufgabe, die das Gerät des Besuchers löst. Cloudflare ist damit kein Empfänger mehr — die Zeilen in AVV und Datenschutz sind gestrichen. Die Nummer A6 bleibt als Abgang stehen, damit ältere Fassungen dieses Verzeichnisses zuordenbar bleiben.

A7

Meta Platforms (WhatsApp, Messenger, Instagram) IRLAND / USA

WhatsApp Cloud API seit dem 14.09.2026 bestätigt in Betrieb; Messenger und Instagram über dieselbe App. Kunden verbinden ihre Nummer per Embedded Signup.

DATEN Telefonnummer, Profilname und vollständiger Nachrichtentext der Endkunden — in beide Richtungen

RANDNOTIZ Genau wegen dieser Daten läuft Inngest auf dem eigenen Server statt in der Cloud (B6).

A8

GitHub USA

Drei Berührungspunkte: „Weiter mit GitHub“ als Anmeldeoption (C7), die Release-Abfrage für die Desktop-App (Server zu Server, der Endnutzer erscheint dort nicht) — und seit dem 17.09.2026 der **Wächter von außen** (0.11): ein Lauf auf GitHubs Maschinen fragt alle 30 Minuten, ob Portal, Marketing-Seite und Zertifikat antworten. Er überträgt nichts als die Frage.

Das Profilbild aus einer GitHub-Anmeldung wird seit demselben Tag **einmal** geholt und liegt danach im eigenen Speicher (0.1) — GitHub sieht die Betrachter nicht mehr.

A9

Let's Encrypt (ISRG) USA · NEU ERFASST

Caddy beantragt **alle** Zertifikate per ACME: `app.lymbe.ai` , `*.lymbe.ai` , die Nebenadressen und über `on_demand_tls` **jede eingetragene Kundendomain**.

Übertragen werden dabei Domainnamen — einschließlich der Domains unserer Kunden und der Mandanten-Slugs —, die Server-IP und die Kontaktadresse `admin@lymbe.ai` . Im Caddyfile ist kein eigener Aussteller gesetzt, es gelten Caddys Vorgaben: Let's Encrypt, ZeroSSL als Rückfall.

BELEG `docker/caddy/Caddyfile — email, on_demand_tls, kein acme_ca`

A10

Spamhaus NEU ERFASST

Der Mailserver fragt für **jede eingehende Mail** die Absender-IP und die enthaltenen Link-Domains bei Spamhaus ab. Das geschieht auch **ohne** DQS-Schlüssel über rspamds eingebaute Regeln; mit Schlüssel zusätzlich über die DQS-Adressen.

Die Abfragen laufen über unseren eigenen Resolver — ihr Inhalt erreicht Spamhaus trotzdem.

B Selbst betrieben — auf der eigenen Maschine

Verarbeiten personenbezogene Daten, aber ausschließlich in unseren Containern. Kein eigener AVV nötig — sie gehören ins Verarbeitungsverzeichnis, nicht in die Empfängerliste.

POS.	KOMPONENTE	ROLLE	INHALT
B1	PostgreSQL (zentral)	Konten, Mandanten, Abrechnung	Name, E-Mail, Passwort-Hash, Rollen
B2	PostgreSQL (je Mandant)	Eine eigene Datenbank pro Kunde	Gespräche, Kontakte, Dokumente, Aufgaben
B3	Qdrant	Vektorspeicher für die Wissenssuche	Textabschnitte aus Dokumenten
B4	MinIO	Dateiablage (S3-kompatibel)	Uploads, Anhänge, Logos, erzeugte Dateien
B5	Redis (2 Instanzen)	Cache, Ratenbegrenzung, Warteschlange	IP-Zähler, zwischengespeicherte Antworten
B6	Inngest (selbst gehostet)	Hintergrundläufe und Crons	Ereignisdaten inkl. WhatsApp-Nachrichten
B7	Caddy	Reverse Proxy, TLS	Zugriffsprotokolle mit IP-Adressen
B8	Pyodide-Sandkasten	Rechnen im Chat	im Container, ohne Netzzugang nach außen
B9	**Mailserver** (Postfix, rspamd, ClamAV, Unbound)	Empfang und Versand der Post für `lymbe.ai`	Absender, Empfänger, Betreff, Text, Anhänge

B10	Fehlerprotokoll und Betriebsprüfung	Fehler und Dienstausfälle im eigenen Haus, alle fünf Minuten geprüft	Fehlermeldung, Ort, Stapel — je Fehlerart eine Zeile

B9

Zum Mailserver im Einzelnen EIGENE MASCHINE

Ein Container auf derselben Maschine, in einem eigenen Netz, mit eigener öffentlicher Adresse nur für Mail. Eingehende Post nimmt Postfix an, rspamd prüft auf Spam und ClamAV auf Schadsoftware, eine interne Brücke reicht sie an die Anwendung. Ausgehende Post signiert rspamd mit DKIM.

Was das ändert: Betreff, Inhalt und Anhänge verlassen unsere Maschine nicht mehr, außer zum Empfänger selbst. Der eigene DNS-Auflöser sorgt dafür, dass auch die Sperrlistenabfragen nicht über einen fremden Resolver gehen — ihr Inhalt erreicht Spamhaus gleichwohl (A10).

Angenommene, aber noch nicht zugestellte Mail liegt in einem eigenen Volume. Dieses Volume und der DKIM-Schlüssel sind **nicht** Teil der nächtlichen Datenbanksicherung; dafür gibt es nur den Server-Snapshot.

C Verbindungen, die der Kunde selbst herstellt

Nicht standardmäßig aktiv. Sie entstehen, wenn ein Kunde im Portal ein Konto verbindet oder eine Adresse einträgt. Die Zugangsdaten lagern bei uns verschlüsselt, die Daten laufen durch unsere Systeme. **C10 und C11 sind beim Nachmessen dazugekommen** — sie beschreiben die Richtung *hinaus*, die bisher ganz fehlte.

POS.	DIENST	UMFANG	DATEN
C1	Google Calendar	OAuth, eigene App	Termine, Teilnehmer, freie Zeiten
C2	Microsoft Outlook / Graph	OAuth, Entra-ID-App	Termine, Kalender, E-Mail je Baustein
C3	Calendly	OAuth + Webhooks	Buchungen, Name, E-Mail

C4	Cal.com	API-Schlüssel + Webhook	wie C3
C5	Shopify	OAuth, Partner-App	Bestellungen, Kundendaten, Sendungsstatus
C6	Shopware / WooCommerce	Admin-API-Schlüssel des Shops	wie C5
C7	Google- und GitHub-Login	OAuth, „Weiter mit …“	E-Mail, Name, Profilbild (siehe 0.1)
C8	Beliebige IMAP-Postfächer	Zugangsdaten, vom Kunden hinterlegt	vollständige E-Mails
C9	Bausteine im Abläufe-Kern	718 Bausteine mit Kontoanbindung	siehe E
C10	**Benachrichtigungs-Ziele**	Slack, Teams oder **freie https-Adresse**	**Name, E-Mail und Nachricht des Besuchers**
C11	**Vom Kunden gesetzte Abrufadressen**	Website-Crawl, API-Import, Logo der Hilfeseite	siehe unten

C10

Benachrichtigungen nach außen NEU ERFASST

Die bisher fehlende Richtung: Lymbe **sendet** Ereignisse an ein Ziel des Mandanten. In der Nutzlast stehen Name, E-Mail und Nachricht des Website-Besuchers, dazu Assistent und Zeitpunkt.

Für Slack und Teams sind die Zieladressen auf `hooks.slack.com` bzw. `*.webhook.office.com` eingegrenzt. Wählt der Kunde „eigene Adresse“, ist **jede** https-Adresse erlaubt; private Netze werden geblockt, Weiterleitungen abgelehnt. Der Empfänger ist damit vom Kunden bestimmt und uns nicht vorab bekannt.

BELEG `inngest/functions/integrations/deliver-event.ts · lib/integrations/destination-domain.ts`

C11

Abrufe an vom Kunden bestimmte Adressen NEU ERFASST

Drei Wege, bei denen unser Server eine Adresse abruft, die der Kunde eingetragen hat:

- **Website-Crawl** für die Wissensbasis, samt `robots.txt` und Sitemap.

- **API-Import einer Wissensquelle** — die konfigurierte Adresse wird mit **entschlüsselten Zugangsdaten** im Header aufgerufen (Bearer, `X-API-Key` oder Basic), ohne Schranke gegen Anfragen ins interne Netz.
- **Logo der öffentlichen Hilfeseite** — liegt kein hochgeladenes Logo vor, wird die eingetragene Adresse unverändert als `<img src>` ausgeliefert. Geprüft wird nur die Länge (max. 500 Zeichen). Der fremde Server erhält damit IP und User-Agent **jedes Besuchers** der Hilfeseite.

D Was im Browser passiert

Der Teil, der Endnutzer direkt betrifft — auf lymbe.ai, im Portal und auf den Websites unserer Kunden. D2 bis D5 sind am 17.09.2026 behoben worden; was das Problem war, steht unter 0.1 bis 0.3 und 0.13. **Seither lädt auf der Website eines Kunden kein fremdes Skript mehr.**

POS.	VORGANG	EINORDNUNG
D1	Chat-Widget auf Kundenwebsites	Speichert auf dem Endgerät — § 25 TDDDG prüfen (0.5)
D2	Sicherheitsprüfung im Widget	**Behoben (0.13)** — Rechenaufgabe von unserem Server; das letzte fremde Skript auf Kundenwebsites ist damit weg
D3	Profilbilder	**Behoben (0.1)** — liegen im eigenen Speicher, kein fremder Abruf mehr
D4	Stripe.js auf der Abrechnungsseite	**Behoben (0.3)** — lädt gar nicht mehr, der Checkout läuft serverseitig
D5	HTML-Mails im Admin-Postfach	**Behoben (0.2)** — Bildsperre, auf Knopfdruck über unsere eigene Route
D6	Marketing-Seite lädt das Demo-Widget	Eigene Domain, gleiche Speichervorgänge wie D1
D7	Schriften	Inter liegt lokal — kein Abruf bei Google Fonts
D8	Baustein-Logos	**Behoben (0.9)** — über `/api/bausteine/[id]/logo` von unserem Server

D8	Baustein-Logos	**Behoben (0.9)** — über `/api/bausteine/logo` von unserem Server
D9	Shopify-Plugin lud Polaris-CSS von `unpkg.com`	**Behoben (0.9)** — ersatzlos entfernt, keine Polaris-Klasse im Markup

D1

Das Chat-Widget § 25 TDDDG

`widget.js` wird von `app.lymbe.ai` geladen, überträgt beim ersten Seitenaufruf den Referrer, legt `lymbe_ignore` im localStorage und Merker für die proaktive Ansprache im sessionStorage ab. Serverseitig dient die IP aus `X-Forwarded-For` der Ratenbegrenzung und Auswertung.

E Der Abläufe-Kern

Fork von Activepieces (Repo `lymbe-ablaeufe`), eigener Container auf demselben Server, eigene Datenbank, eigener Redis-Bereich. Getrennt zu betrachten. Alle Werte am 17.09.2026 im laufenden Container nachgemessen.

POS.	PUNKT	STAND
E1	Telemetrie	`AP_TELEMETRY_ENABLED=false` — geht nicht hinaus
E2	Katalogabgleich	**Behoben (0.10)** — `AP_PIECES_SYNC_MODE=NONE`, kein Abgleich, kein Abruf nach außen
E3	Verbundene Bausteine	rund 765 im Katalog, 718 mit Kontoanbindung — je Baustein ein eigener Empfänger, sobald ein Kunde ihn verbindet
E4	Aufbewahrung und Absicherung	Laufdaten 30 Tage, Dateien max. 25 MB, Registrierung aus, `SANDBOX_CODE_ONLY`, `AP_NETWORK_MODE=STRICT`

Dieselbe Frist von 30 Tagen gilt für die nächtlichen Datenbanksicherungen: gelöschte

Kundendaten und alte Schlüssel verschwinden damit spätestens nach 30 Tagen auch aus den Sicherungen. Eine wöchentliche Probe spielt eine Sicherung wirklich zurück.

F Bezugsquellen — Lieferkette ohne Kundendaten

Hierhin gehen keine personenbezogenen Daten, wohl aber unsere Server-IP — und von dort kommt Code und kommen Daten herein. Für ein Verarbeitungsverzeichnis nachrangig, für die Frage „was läuft auf dieser Maschine“ nicht.

POS.	QUELLE	WANN	WAS
F1	`registry.npmjs.org`	nur beim Bauen	Abhängigkeiten, Baustein-Pakete
F2	`cdn.activepieces.com`	höchstens einmal je Logo und Tag	Baustein-Logos, **vom Server** geholt (D8)
F3	`database.clamav.net`	täglich	Virensignaturen, rund 170 MB
F4	`cdn.jsdelivr.net` , `pypi.org`	nur beim Bauen	Pyodide-Pakete für den Sandkasten — zur Laufzeit keine Verbindung

G Im Projekt vorhanden, aber nicht geschaltet

Diese Wege sind im Code vorgesehen und mit Zugangsdaten versehen, aber keiner ist aktiv. Sie stehen hier, weil ein Prüfer sie fände — und weil ein Anbieterwechsel **kein Deploy-Ereignis** ist, sondern eine Zeile in einer Konfigurationsdatei.

POS.	EINTRAG	STAND

G1	Azure OpenAI	Zugangsdaten gesetzt, `AI_PROVIDER=stackit` — kein Empfänger
G2	Mistral AI	dito, Rückfallebene
G3	Activepieces Enterprise-Module (`ee/`)	Verweise auf PostHog, Cloudflare-SaaS und AppSumo — wir fahren `AP_EDITION=ce`, die Module laufen nicht

H Was bewusst nicht eingesetzt wird

Nachweisbar aus dem Code — die Gegenprobe zur Aussage in der Datenschutzerklärung.

- **Keine Analyse- oder Tracking-Werkzeuge.** Weder Google Analytics noch Plausible, Matomo oder PostHog. Die einzige Auswertung ist unsere eigene, in der Mandanten-Datenbank.
- **Keine Werbe- oder Remarketing-Pixel.** Kein Meta-Pixel, kein LinkedIn Insight Tag, kein Google Ads Tag. Die Verweise auf `facebook.com` und `linkedin.com` sind Links im Fußbereich.
- **Kein Tag Manager** und kein nachladbares Skript-Gerüst.
- **Kein Dienst für den Mailversand.** Empfang und Versand laufen auf eigener Maschine (B9) — kein Anbieter sieht Betreff, Inhalt oder Anhänge.
- **Kein Fehler-Monitoring bei Dritten.** Das Fehlerprotokoll liegt in der eigenen Datenbank, ohne Sitzungsaufzeichnung (B10).
- **Keine Suchmaschinen-API.** Kein Brave, Tavily oder SerpAPI — die Websuche nutzt die eingebaute Suche des Modellanbieters, um keinen weiteren Vertragspartner mit Einblick in die Fragen der Mitarbeitenden zu schaffen (A4).
- **Keine Schriften von fremden Servern** (D7).
- **Nur technisch notwendige Cookies:** Sitzungs-Cookie und CSRF-Token. Stripe setzt im Browser eigene (A5).

Wie diese Liste entstanden ist. Ausgewertet wurden die Abhängigkeiten beider Repositories, die Umgebungsvariablen in `.env.example` und im Produktions-Compose, sämtliche ausgehenden Hostnamen im Quellcode, alle Skript- und Font-Einbindungen sowie die Speichervorgänge in `widget.js` . Schalterstellungen, Standort und Typ der Maschine, DNS-

Einträge und die Einstellungen des Abläufe-Kerns sind am laufenden System gemessen, nicht aus der Dokumentation übernommen.

Was sie nicht leisten kann. Der Quellcode zeigt, welche Wege möglich sind — nicht, welche morgen geschaltet sind (Abschnitt G). Und sie sagt nichts über den Vertragsstand: welche AVV tatsächlich geschlossen sind, ist hier nicht geprüft.